



Contenido

01

¿Qué es una clase en Java?

02

¿Qué es un objeto?

03

Atributos en Java

04

Métodos (Comportamiento)

05

Crear una clase en Java

06

Creación de objetos

07

Abstracción

08

Abstracción para crear una clase

09

Caso práctico: Animales

10

Errores Comunes en clases

11

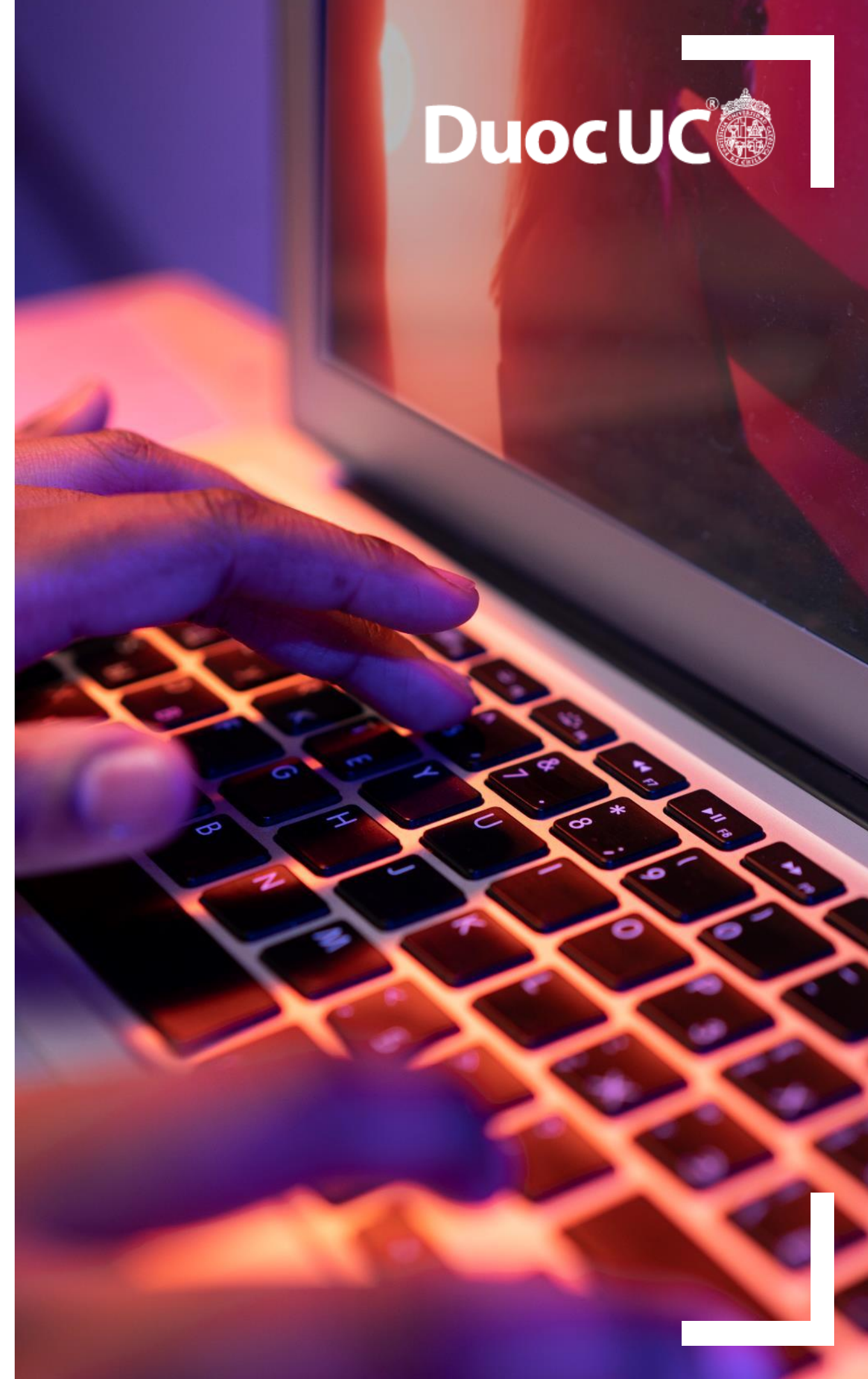
Buenas prácticas

12

Resumen visual clases y objetos

13

Conclusiones y próximos pasos



</> Fundamentos del Lenguaje

Clases y Objetos en Java

Atributos, métodos y abstracción.

**Clases (Plantilla)**

Animal

**Objetos (Instancia)**

Perro, Gato

```
ZooApp.java
// 1. Definición de Clase
class Animal {
    // Atributos (Estado)
    String nombre;
    int edad;

    // Métodos (Comportamiento)
    void hacerSonido() {
        System.out.println("Sonido de animal");
    }
}

// 2. Creación de Objetos
Animal miPerro = new Animal();
miPerro.nombre = "Max";
miPerro.hacerSonido();
```





¿Qué es una Clase en Java?

El plano o molde fundamental de la Programación Orientada a Objetos



Definición

Una **clase** es una plantilla o diseño a partir del cual se crean objetos individuales, definiendo sus características comunes.



Plantilla o Molde

Es un concepto general (ej. Animal), no un ente específico.



Define Estructura

Establece qué datos (atributos) y acciones (métodos) tendrán sus objetos.



Agrupar Datos y Comportamiento

Encapsula la información y las operaciones relacionadas en un solo lugar.

Concepto Clave:

-  Sin clases, Java no funciona; todo el código debe estar dentro de una clase.

El Plano: Clase Animal



Clase: Animal

Atributos (Datos)

- nombre: String
- edad: int

Métodos (Acciones)

- + comer()
- + hacerSonido()



¿Qué es un Objeto?

Un objeto es una instancia concreta creada a partir del molde que define una clase.



La Clase (El Molde)

Define los atributos (qué datos tiene) y métodos (qué puede hacer) de forma general.



El Objeto (La Instancia)

Es una entidad real que existe en la memoria. Tiene sus propios valores específicos para los atributos definidos en la clase.



Creación (Instanciación)

En Java, se usa la palabra clave `new` para crear un nuevo objeto.

Ejemplo: De la clase `Animal` podemos crear muchos animales distintos, cada uno con su propio nombre y edad.

DE LA CLASE A LOS OBJETOS

Clase: `Animal`

```
String nombre;  
int edad;  
hacerSonido()
```

perro1

nombre: "Max"
edad: 5

"¡Guau!"

gato1

nombre: "Luna"
edad: 2

"¡Miau!"

pato1

nombre: "Donald"
edad: 1

"¡Cuac!"



Atributos en Java

Definiendo el estado y las características de los objetos

¿Qué son?

Los atributos son las variables que se declaran dentro de una clase. Su propósito principal es guardar los datos y definir el **estado** de un objeto en particular.



Estado del Objeto

Representan las propiedades únicas que diferenciarán a un objeto de otro de la misma clase.



Ejemplos Comunes

En una clase `Animal`, los atributos pueden ser: *nombre*, *edad*, *peso* y *color*.

De la Clase al Objeto

Clase: Animal (Plantilla)

```
String nombre;  
int edad;  
double peso;  
String color;
```



INSTANCIACIÓN (new)

Objeto: perro1 (Instancia)

```
nombre = "Max";  
edad = 3;  
peso = 15.5;  
color = "Café";
```



▶ Métodos (Comportamiento)

Las acciones que los objetos pueden realizar

🏃 ¿Qué hacen los objetos?

Mientras los atributos describen cómo es el objeto (su estado), los **métodos** definen las acciones que puede llevar a cabo (su comportamiento).

🔊 **hacerSonido()**
El animal emite un ruido característico.

🍴 **comer()**
El animal consume alimento y recupera energía.

💤 **dormir()**
El animal descansa durante un tiempo.

Clase Animal

🐾 Objeto: Perro

Métodos Ejecutados

- hacerSonido() → "Guau!"
- comer() → Croquetas
- dormir()

🐾 Objeto: Gato

Métodos Ejecutados

- hacerSonido() → "Miau!"
- comer() → Atún
- dormir()

Main.java

```
// Llamando métodos en los objetos
Animal miPerro = new Animal("Max");
Animal miGato = new Animal("Luna");

miPerro.hacerSonido(); // Acción para el perro
miGato.comer(); // Acción para el gato
```





Crear una Clase en Java

Estructura básica: class, atributos y métodos



¿Cómo se define?

Para crear una clase usamos la palabra clave `class`, seguida del nombre (en mayúscula inicial) y su contenido encapsulado entre llaves `{}`.

1. Declaración de Clase

Define el nombre de la plantilla. Todo el código de la clase vive dentro de su bloque.

2. Atributos (Estado)

Variables declaradas a nivel de clase. Cada objeto creado tendrá su propia copia de estos datos.

3. Métodos (Comportamiento)

Funciones internas que definen las acciones que los objetos de esta clase pueden realizar.

Estructura: class Animal

```
class Animal {  
  
    // Atributos  
    String nombre;  
    int edad;  
  
    // Métodos  
    void hacerSonido() {  
        System.out.println("¡Hace un sonido!");  
    }  
}
```



Este código define la plantilla abstracta. Aún no se ha creado ningún animal real (objeto) en la memoria.



Creación de Objetos

Instanciación usando new y asignación de atributos

¿Qué es Instanciar?

Para utilizar una clase, debemos crear un objeto (instancia) real en la memoria utilizando la palabra clave **new**.



1. Declaración

Crear una variable del tipo de la clase que servirá como referencia.



2. Instanciación

Usar new para asignar la memoria necesaria para el nuevo objeto.



3. Uso (Punto .)

Utilizar el operador punto . para acceder a atributos y métodos.

Dato Importante:

Múltiples objetos instanciados desde la misma clase son totalmente independientes entre sí.

Flujo de Creación Paso a Paso



Paso 1: Declarar e Instanciar

```
// Tipo Nombre = new Clase()  
Animal perro = newAnimal();
```



Paso 2: Asignar Atributos

```
// Rellenando los datos del objeto específico  
perro.nombre = "Max";  
perro.edad = 3;
```



Paso 3: Invocar Métodos

```
// Ejecutando comportamientos del objeto  
perro.hacerSonido(); // Salida: "Guau!"  
perro.comer();
```



Abstracción

Mostrar solo lo esencial de una entidad

¿Qué es Abstraer?

La **abstracción** consiste en identificar las características y comportamientos clave de un objeto del mundo real para modelarlo en software, ignorando los detalles que no importan para nuestro sistema.



Lo Esencial (Nos Importa)

Para un veterinario: Nombre, Edad, Especie, hacerSonido().



Lo Irrelevante (Descartamos)

Para un veterinario: Color favorito de juguete, historial de ladridos a las 3 AM.



Enfoque Claro

Simplifica el problema y hace el código más fácil de entender y mantener.

Proceso de Abstracción



Animal Real

- ✓ Nombre: "Max"
- ✓ Edad: 3 años
- ✗ Comió a las 8am
- ✗ Pelo corto
- ✗ Le teme a los truenos
- ✓ Puede ladrar



Clase Animal

```
class Animal {  
    String nombre;  
    int edad;  
    void hacerSonido() { }  
}
```



Abstracción para Crear una Clase

Pensar qué datos y comportamientos son importantes

El Proceso de Abstracción

La **abstracción** ayuda a diseñar clases enfocándose solo en los detalles relevantes para el programa, ignorando la complejidad innecesaria de la entidad real.

-  **Observar la Realidad:** Un animal tiene millones de células, pero en la app solo importa su nombre y edad.
-  **Seleccionar Atributos:** Elegimos datos clave como 'nombre', 'edad' o 'raza'.
-  **Definir Métodos:** Seleccionamos acciones útiles como 'hacerSonido()' o 'comer()'.

Regla de Oro:

-  **Abstraer es simplificar la realidad para convertirla en código útil.**

Ejemplo Visual: Animal Real vs Clase Java



Animal Real

Millones de células
Pelaje complejo
Historia genética
Dieta específica



Abstracción Filtrar lo esencial



Animal (Clase)

- nombre: String
- edad: int

+ hacerSonido()
+ comer()

Ejemplos: Modelando clases específicas

Perro (Clase)

```
String raza;  
void ladrar() {  
    print("Guau");  
}
```

Gato (Clase)

```
boolean esDeInterior;  
void maullar() {  
    print("Miau");  
}
```



Caso Práctico: Animales

Creando y usando una clase Animal con objetos reales

Instanciando Animales

En un programa real, creamos una clase como plantilla y luego generamos múltiples objetos a partir de ella, reutilizando su estructura.

 **Clase Animal**
Plantilla general con nombre, edad y hacerSonido().

 **Objeto Perro**
Instancia creada usando `new Animal()`.

 **Objeto Gato**
Segunda instancia concreta de la misma clase.

Diagrama de Clase e Instancias



```
Animal perro = newAnimal("Max", 3);
Animal gato = newAnimal("Luna", 2);
// Reutilizando la misma plantilla para ambos objetos
```



Errores Comunes en Clases

Problemas frecuentes al modelar y crear objetos



¿Qué suele salir mal?



Mezclar Clase con Objeto

Intentar asignar valores directamente al molde (la clase) en vez de a la instancia.



Olvidar Atributos Esenciales

Crear clases sin el estado mínimo necesario para que funcionen correctamente.



Demasiadas Responsabilidades

Hacer que una sola clase 'Animal' gestione también la base de datos y la interfaz gráfica.



Nombres Poco Claros

Usar nombres como 'A1' o 'Dato' en lugar de 'Perro' o 'edad'.



Error Consecuencia Solución

Asignar valor a la Clase

```
Animal.nombre = "Max";  
// Error si no es static
```

→ Error de compilación

```
Animal perro = new Animal();  
perro.nombre = "Max";
```

Clase 'Dios' (Todo en uno)

```
class Animal {  
    void comer() { ... }  
    void guardarEnBD() { ... }  
}
```

→ Difícil de mantener

```
Separar: class Animal y  
class AnimalRepositorio
```

Nombres ambiguos

```
class A1 {  
    int d;  
}
```

→ Código ilegible

```
class Perro {  
    int edad;  
}
```



Buenas Prácticas

Consejos para diseñar clases limpias, cohesivas y fáciles de mantener

Reglas de Oro

Una Sola Responsabilidad

Una clase debe tener una única razón para cambiar. (ej. `GestorAnimal`, NO `AnimalBaseDeDatosYUI`).

Nombres Coherentes

Clases: Sustantivos (PascalCase). Métodos: Verbos (camelCase). Atributos: descriptivos (camelCase).

Atributos Significativos

Incluye solo el estado que realmente define al objeto en el contexto de tu aplicación.

Abstractar lo Importante

Ocultar los detalles de implementación complejos detrás de métodos públicos claros.

Checklist Visual: Lo que SÍ debes hacer



Encapsula por Defecto

```
private String nombre; // Mejor que public String nombre;
```



Usa Constructores para Inicializar

```
Animal perro = new Animal("Max", 3);
```



Sobrescribe toString()

Para facilitar la depuración y lectura del objeto.



Favorece la Composición sobre la Herencia

Usa 'tiene un' en lugar de 'es un' cuando tenga sentido.



Resumen Visual: Clases y Objetos

Hoja de repaso rápido de la estructura orientada a objetos (UML simplificado)



Conceptos Clave

Clase

La plantilla o plano principal que define la estructura general (ej. `Animal`).

Objeto

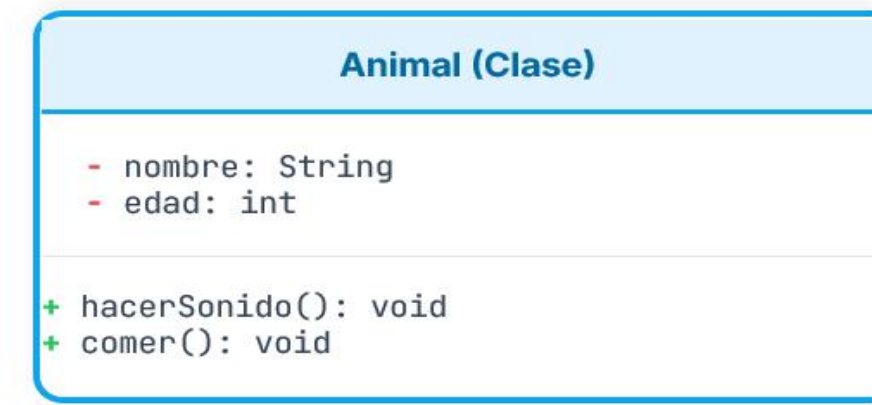
La instancia real creada en memoria a partir de la clase (ej. `perro1`).

Atributos

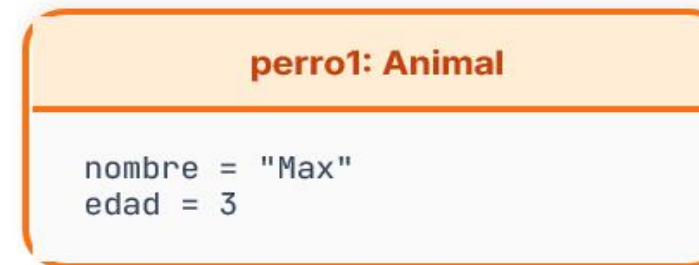
Variables que almacenan el **estado** o datos específicos del objeto.

Métodos

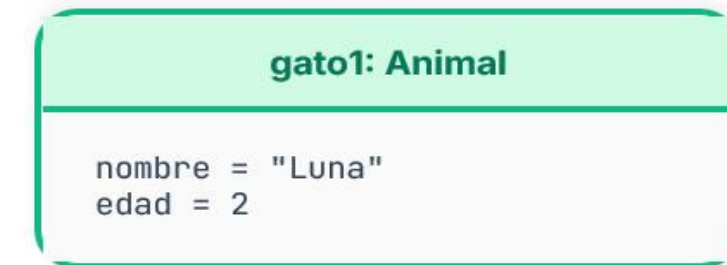
Funciones que definen el **comportamiento** y las acciones que puede realizar.



Instancia (new)



Instancia (new)



 **Abstracción en acción:** Diseñamos la clase `Animal` capturando solo los atributos y comportamientos esenciales, permitiendo reutilizar esa estructura para crear múltiples objetos con sus propios valores únicos.



Conclusiones y Próximos Pasos

Cierre de Clases y Objetos en Java

Ideas Clave

Comprender el uso de **clases y objetos** es fundamental para modelar la realidad y crear software estructurado.



Clase y Objeto

La clase es el modelo o plantilla; el objeto es la instancia viva en memoria.



Atributos y Métodos

Los atributos describen el estado; los métodos actúan y definen el comportamiento.



Abstracción

Seleccionar solo lo esencial ayuda a diseñar mejor y reducir la complejidad.

Resultado:

✓ **Código más organizado y fácil de entender**

Visión General de Clases y Objetos



→ Próximos Pasos

1

Constructores

2

Encapsulamiento

3

Herencia



Clases y Objetos en Java



Buenas Prácticas



Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación